



Universidad
Indoamérica

ユニット 9・オペレーティングシステムの基礎

モデルが提案し、 オペレーティングシステムが 決める

Universidad Tecnológica Indoamérica

AI ネイティブ OS

チャットからエージェントへ

LLM

テキスト → 次のテキスト。

エージェント

テキスト → **行動**（ツール） → **結果** →
テキスト...

- ユーザの代わりに動くが、**ユーザ本人ではない**。
- 自信満々に間違えることも、だまされることもある（プロンプトインジェクション）。

5つの責任、3つのプラットフォーム

delete old_report.pdf

TD8 (このコース)

ユーザ

エージェント (LLM)

ツール呼び出し

OS : 決める

ファイル / アプリ

エージェントの ID

UNIX ソケット 0600 + SO_PEERCRED (クライアントの uid と pid)

ツール (コネクタ)

11 個のツールの閉じたカタログ、厳密なパーサ

同意

agentd は delete_file の前に尋ねる

隔離

agent_exec の fork + exec : setrlimit、alarm、Landlock

監査

O_APPEND の audit.log : 判断ごとに 1 行

エージェント型 OS としての Windows

- エージェント専用アカウントを持つ **Agent workspace**（プレビュー）
 - **Agent connectors**：**On-Device Registry** 内の MCP サーバ（Insiders、2025 年 12 月）
 - 原則：見分けられる行動、ユーザによる承認、最小権限
 - **Microsoft Execution Containers**（Build 2026）：プロセス・セッション・マイクロ VM
- 出典と日付：マニュアル M9 §2。

Linux：組み立てる部品

Landlock

パスとネットワーク、root
なしで

seccomp

どの呼び出しか

namespaces

・cgroups

何が見えるか・どれだけ使うか

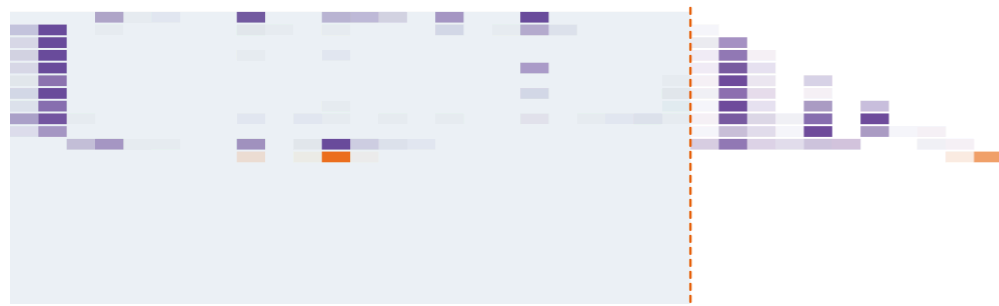
ディストリビューション：Ubuntu は *inference snaps* と隔離されたエージェントを発表（26.10 で
プレビュー）。ハードウェア：NPU のための `accel` サブシステム。

LLM の内側：バイト、KV キャッシュ、アテンション

要求と答えを1バイトずつ

Q : _ s h o w _ m e _ n o t a s . t x t ↩ A : _ r e a d _ f
i l e ("

アテンションマップ：ステップごとに1行、バイトごとに1列



次のバイト：確率の高い5つ

n	100 %
w	0.0 %
y	0.0 %
p	0.0 %
-	0.0 %

```
read_file("█
```

C で書いた 174,656 パラメータを OS が測る

実際の出力

```
$ ./tinylm -v "lista los archivos de docs"
""
list_files("docs")

model      : 174656 parameters, 3 layers, 4 heads, d=64, context 128 bytes
weights    : 698688 bytes mapped with mmap (MAP_PRIVATE, PROT_READ)
KV cache   : 201984 bytes per request (+ scratch)
tokens     : 33 of prompt (prefill) + 19 generated (decode)
time       : load 0.042 ms, inference 9.08 ms (5730 tokens/s)
page faults: 2 during load, 47 during the first inference (minor; major: 0)
```

- 重みは**マップ**される（ユニット 6）：読み込みは 0.1 ミリ秒単位で、ページフォールトは 2 回。残りは使うときに来る（ユニット 4）
- KV キャッシュは**要求ごとの**メモリ。重みは全員で共有する

99 % 正しくても十分ではない

v1

テストでは完全一致率 99.4 %...だが演習では
`viejo.txt` を `viefe.txt` に変えてしまった：
そういう名前をほとんど見ていなかった。

v2

同じ 3 層のモデル (174,656 パラメータ) だが、
より多様な名前を含むデータと、10,000
ではなく 12,000 ステップ：新しい 500 件の
要求で 99.0 %、珍しい名前も正しく写す。

何千もの行動で 1 % の誤りは、何十もの間違った行動になる：だから OS はモデルの出力を信用しない (§5)。

OS から見た推論

部品	コースのユニット
<code>mmap</code> した重み、共有	U6
重みに触れたときのページフォールト	U4
トークンごとに大きくなる KV キャッシュ	U4
プール内の、要求ごとに1スレッド	U7
デバイスとしての NPU / GPU	U1・U3
エージェントはローカルソケットで <code>agentd</code> と話す	U8

OS の負荷としての推論

micro-LLM (TD8)・実測

tinyllm のコード

ヒープ: KV キャッシュ + 作業領域

52 / 128 位置・78.0 KiB

mmap: 重みファイル (読み取り専用)



171 4 KiB のページ・使用時のフォールト 47

スタック

エージェント 8 個分のメモリ

共有された重み (mmap)



各エージェントが重みをコピー



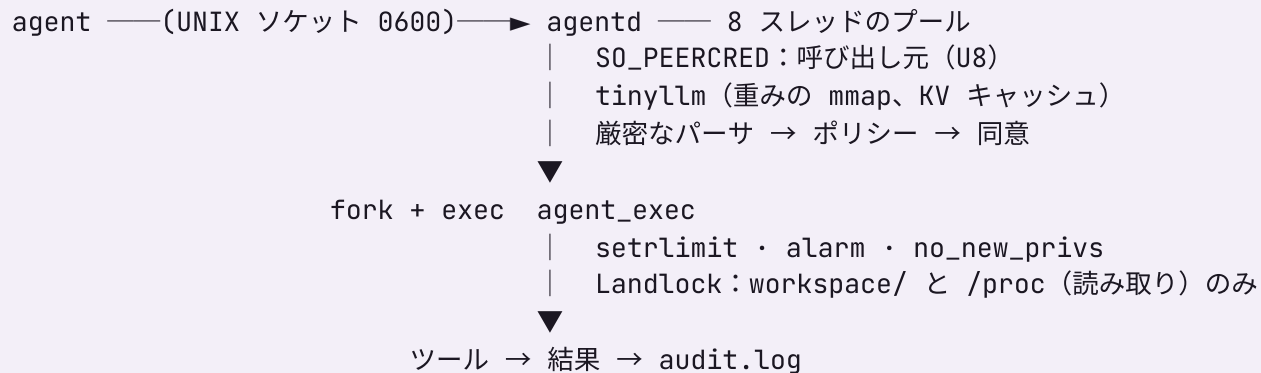
■ 重み ■ KV キャッシュ

1 つの要求: 入力 33 トークン (prefill) + 生成 19・合計 9.08 ms



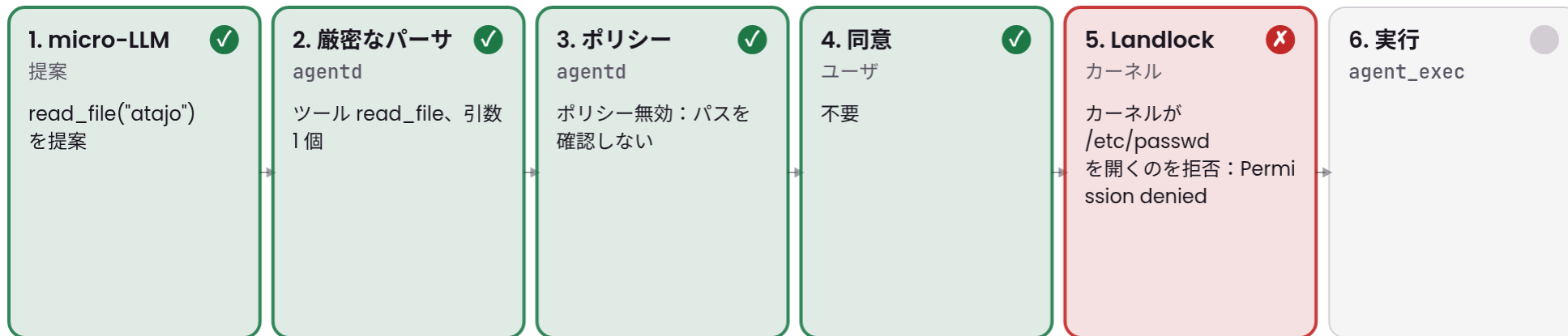
1 コアでトークンあたり約 175 μ s

コースのエージェント型 OS



多層防御

要求: read atajo



agentd が audit.log に追記する行:

```
2026-09-25T10:00:00 uid=1000 pid=4242 request="read atajo" call=read_file("atajo") decision=deny outcome=error (Permission denied)
```

TD8 + AI チャレンジ

Bash

```
# 要求は英語で: micro-LLM が理解するのはスペイン語と英語だけ
cd tds/bin/td8 && mkdir -p workspace && echo hello > workspace/notes.txt
./agentd -w workspace &
./agent "show me notes.txt"
./agent "delete /etc/passwd"
cat audit.log
```

チャレンジ: `copy_file` を加え（データ、再訓練、ポリシー、実行器）、5 つの攻撃を考える。



Universidad
Indoamérica

まとめ

3つの問い

1. エージェントは OS にどんな 5 つの責任を求めるか？
2. なぜモデルの出力は「信用できないテキスト」なのか？
3. ポリシーが失敗したとき、ずるいリンクを止めるのはどの層か？